COMPUTER SCIENCE + COMPUTER ENGINEERING

06

Engine Reference and Code Map

Conventions, formulas, formats and implementation boundaries

CPU/GPU contracts · GEM1/GET1 formats · Source map · Actual validation scope

PACKAGE 06 / EXPANDED VOLUME 05 / SEPTEMBER 28, 2026
Self-study: derive, implement, test and explain.

Engine Reference and Code Map

Use this as a contract sheet, not a substitute for the lessons

This companion records the conventions actually used by the shipped examples. A native GPU implementation must make its own API-specific rules explicit. The small helper exercises are models of decisions, not a full engine implementation.

Coordinate and interpolation sheet

Item	CPU reference convention
View space	Right-handed; camera looks toward -Z
Transform convention	Column vectors; projection maps view to homogeneous clip
Clip inequalities	-w <= x <= w; -w <= y <= w; 0 <= z <= w
Projection domain	0 < near < far; positive aspect; 0 < vertical field of view < pi
Perspective divide	Only after clipping, with positive finite w
NDC to raster	x=(x_ndc/2+1/2)*width*; *y=(1/2-y_ndc/2)*height
Sample center	(x+0.5,y+0.5); screen origin upper-left
Subpixel grid	1/256 pixel; edge tests use signed 64-bit integer arithmetic
Coverage sign	Normalize triangle area positive in screen coordinates
Inclusive edge	dy<0, or dy==0 and dx>0, under that sign/y-down convention
Depth	Clear1, strict less; projected depth affine in screen barycentrics
Material	Linear vertex color times point-sampled procedural checker
Output	P6 RGB8, opaque, upper-left origin, explicit sRGB-coded variant

For a right-handed camera looking down -Z and a [0,1] depth interval, let s=1/tan(fovy/2), A=far/(near-far), B=far*near/(near-far). Then clip=(*s*x/aspect, *s*y, *A*z+B, -z). At view z=-near, projected depth is 0; at z=-far it is 1.

For a clip plane distance d, a crossing edge from A to B intersects at t=d(A)/(d(A)-d(B)); linearly interpolate the full homogeneous position and raw attributes at that t. Do not interpolate already divided coordinates and call it equivalent.

For screen barycentrics lambda_i, ordinary varying a is:

```
a = sum(lambda_i * a_i / w_i) / sum(lambda_i / w_i)
```

The depth exception for this baseline is z_ndc=sum(lambda_i*z_clip_i/w_i), without a second quotient. With weights (.2,.3,.5), attributes (0,1,1) and clip w=(1,2,4), the perspective-correct scalar is 11/19, not the affine .8.

A half-open edge rule belongs to a particular sign/origin convention. Copying it into a y-up rasterizer without rederiving the orientation can create cracks or double coverage. Quantization and sample positions should be included in CPU/GPU comparison tolerances. [R19, S01]

Color and material sheet

For an encoded sRGB channel s in [0,1], decode with s/12.92 when s<=0.04045; otherwise ((s+0.055)/1.055)^2.4. For a linear channel l in [0,1], encode with 12.92*l when l<=0.0031308; otherwise 1.055*l^(1/2.4)-0.055. The reference clamps its LDR output; this does not implement HDR tone mapping.

Average color in the declared linear domain before encoding. A black/white pair averages to linear 0.5, which encodes to approximately0.7354. Normal, roughness, metallic, depth and ID maps have data semantics and should not automatically receive color decoding. PPM's strict specification differs from the common sRGB variant; the reference explicitly labels its variant in the comment. [S02, S03]

For a passive Lambertian surface f_r=rho/pi. If incident irradiance E already includes angular integration/cosine effects, outgoing radiance is rho*E/pi. Do not multiply by the same cosine twice. A roughness parameter must be interpreted under its selected microfacet model; a shiny appearance alone does not establish physical correctness. [S13, S14]

A complete square power-of-two mip chain approaches 4/3 of the base texel count. A 256-by-256 chain has 87,381 texels including the base and 1-by-1 level. Non-square/odd dimensions must be counted explicitly under the halving policy. The cooker floor-halves each extent, uses an area-weighted linear-light box filter and includes the entire source area even when dimensions are odd. [S15]

Submission, context and presentation sheet

Submission return is not execution completion. Track producer/consumer queue, submission, stage, access and range before choosing synchronization. CPU wrapper lifetime and GPU resource lifetime are distinct. A frame context becomes reusable only after every relevant last consumer has completed under that API's rules. [S05-S09]

Concern	Vulkan	Direct3D 12	Metal	WebGPU bridge
Command recording	Command buffers	Command lists/ allocators	Command buffers/ encoders	Command encoders/ passes

Concern	Vulkan	Direct3D 12	Metal	WebGPU bridge
Dependencies	Stages/accesses, barriers and submission synchronization	Resource states/ barriers plus queue fencing	Use native tracking, storage and synchronization rules	Validated usages; transitions managed by API
CPU reuse	Relevant fence/ timeline completion	Relevant fence completion	Relevant command completion/resource rules	Per-slot submitted-work promise
Presentation	Swapchain acquire/ present and image-specific lifetimes	Swapchain and platform rules	Drawable/platform rules	Canvas context-managed current texture

This is a conceptual comparison, not a flag-by-flag translation. The lesson's Vulkan compute-to-vertex-input example is not identical to the browser demo's compute-to-vertex-shader storage read. Use the actual consumer access, not whichever sample barrier looks familiar. [S07, S09, S10]

In Vulkan, a graphics submission fence alone may not establish that presentation finished waiting on its render-finished semaphore. The current tutorial/guide describe an image-indexed semaphore-reuse scheme. Do not confuse frame-slot index with acquired swapchain-image index. During acquire/recreate handling, avoid resetting a fence and then taking an early path that submits nothing to signal it. [S05, S08, S12]

Shipped source map and boundaries

Source	Implemented here	Learner extension
code/cpp/renderer.hpp	Six-plane attributed clipping, fixed-point coverage, depth, perspective interpolation, PPM, GEM1 input	General meshes/materials, image sampler, filters, ray visibility integration
code/cpp/main.cpp	Bounded CLI extent, built-in or cooked scene, CPU render-only timing	Repeated controlled benchmark, live engine loop
code/cpp/tests.cpp	200 assertions in one test executable	More property/fuzz/metamorphic and cross-backend tests
exercises/starter	20 deliberate NotImplementedError contracts	Your implementations and transfer cases
exercises/reference	Selected pure decision/arithmetic/ state helpers	Actual entities, world, scheduler, resource systems
code/python/bvh_reference.py	Median-split sphere BVH and brute-force oracle	Triangle BVH, update policy, build/ query crossover analysis

Source	Implemented here	Learner extension
code/python/cook_assets.py	Validated JSON sources, deterministic GEM1/GET1 payloads, dependency keys, atomic publication	General importers, real compression/ targets, runtime GET1 sampling
code/webgpu/demo.js	Source-only bounded transform/ readback and cube drawing with two contexts	Hardware validation, native backend, complete engine integration
validate_all.py	Reproducible CPU test/build/cook/ render runner	Your complete Gate E evidence automation

The Python entity exercise checks handle validity; it does not supply a complete entity allocator. The topology exercise validates ordering; it is not a parallel scheduler. The memory-admission exercise is a combined-accounting arithmetic model; it is not a real residency manager. The edit exercise records a command and clears redo, but a live inspector and full undo/redo dispatcher are assigned. The scene validator migrates a small schema; crash-durable save/load and asset resolution require further work.

Native teaching file formats

GEM1: cooked nonindexed triangles

Little-endian, no implicit struct padding. Bytes0-3 are ASCII GEM1. Bytes4-7 are uint32 vertex count. The count must be nonzero, divisible by3 and at most300,000. Each vertex then occupies32 bytes: eight IEEE754 binary32 values in order x,y,z,u,v,r,g,b. Positions are view-space values for the current demo; w is supplied as1 before projection. Colors are linear [0,1]. The teaching parser rejects nonfinite components, absolute values above10,000, invalid colors, truncated payloads and trailing bytes. This bound is a deliberate lab policy, not a universal geometry limit.

The C++ reader decodes the little-endian words explicitly and bit-casts to float; it does not cast disk bytes to a host struct. The supplied mesh contains12 vertices, so the payload is8+12*32 =392 bytes. [Original package format]

GET1: cooked linear RGB mip chain

Little-endian. Bytes0-3 are ASCII GET1. Bytes4-7 are base width uint32; bytes8-11 base height; bytes12-15 level count. Each level contains uint32 width, uint32 height, then width*height consecutive RGB triples of binary32 linear-sRGB values, 12 bytes per texel. Levels include the base and end at1-by-1. Source dimensions are limited to1..256 for this teaching cooker.

An8-by-8 chain has85 texels across four levels. Its payload is16 header bytes *+48 level-header bytes +85*12 data bytes =1,068 bytes. The final checker average is linear 0.5. The current C++ renderer does **not** load GET1; add a checked runtime reader and sampler as part of the asset/texture lab, or use your documented alternative runtime format.

Content-addressed manifest

assets_manifest.json maps stable logical names checker and scene to derived cache keys, filenames, payload digests and byte counts. The image key includes source digest, canonical settings and tool version.

The mesh key also includes its logical checker dependency key. Key and payload hash serve different purposes: a cache key names the build contract; the payload hash verifies the emitted bytes.

The cooker validates/builds both payloads before publishing a new manifest and writes via a temporary file plus replacement in the same directory. It fsyncs the temporary file but does not claim full cross-platform directory/power-loss durability. Publishing two derived files then replacing the manifest is a useful teaching transaction boundary; production rollback, garbage collection and multi-writer concurrency require additional design.

World and command exercise schemas

Scene version1 uses an entities list whose entries contain a unique nonempty string id and position object {x,y,z}. Version2 replaces that position with a three-number list. The validator returns a new version2 candidate, retains only defined fields and rejects unsupported versions, duplicates, nonfinite positions and values outside +/-1,000,000. It never mutates the supplied document. This is a deliberately narrow schema, not a general asset/world serializer.

The edit exercise receives a dict mapping entity ID to a position list plus undo/redo lists. A valid non-no-op edit appends {id,before,after}, publishes the new position and clears redo. Invalid inputs leave state/history unchanged. A no-op returns false and leaves history intact. The full command dispatcher, entity deletion semantics and authoring/simulation boundary remain learner work.

Useful budget identities

Fixed step: with accumulated elapsed T and step dt, full ticks=floor(T/dt), remainder=T mod dt. The supplied bounded helper executes at most max_steps, drops excess whole ticks under its declared policy and retains the fractional remainder. It reports dropped time explicitly. That overload choice is not universal. [R27]

An uncompressed attachment payload is width*height*bytes_per_pixel*samples. At1920-by-1080 and16 bytes/pixel, one single-sample attachment set is33,177,600 bytes; 4x samples multiply that raw payload by4. This excludes tiling/metadata/padding and does not directly equal bandwidth or total residency.

At an arena offset17, an8-byte request aligned to16 begins at32, consumes15 bytes of padding and ends at40. Actual arena implementation and GPU-consumer reset guards are exercises; the package's topology helper does not allocate an arena.

Nearest-rank p percentile on N sorted samples uses one-based rank ceil(p*N/100), for0<p<=100. The supplied function requires finite nonnegative timings and a nonempty sample list. Report the quantile convention and retain raw values; a percentile from five frames does not characterize an engine's stable tail.

Validation actually performed

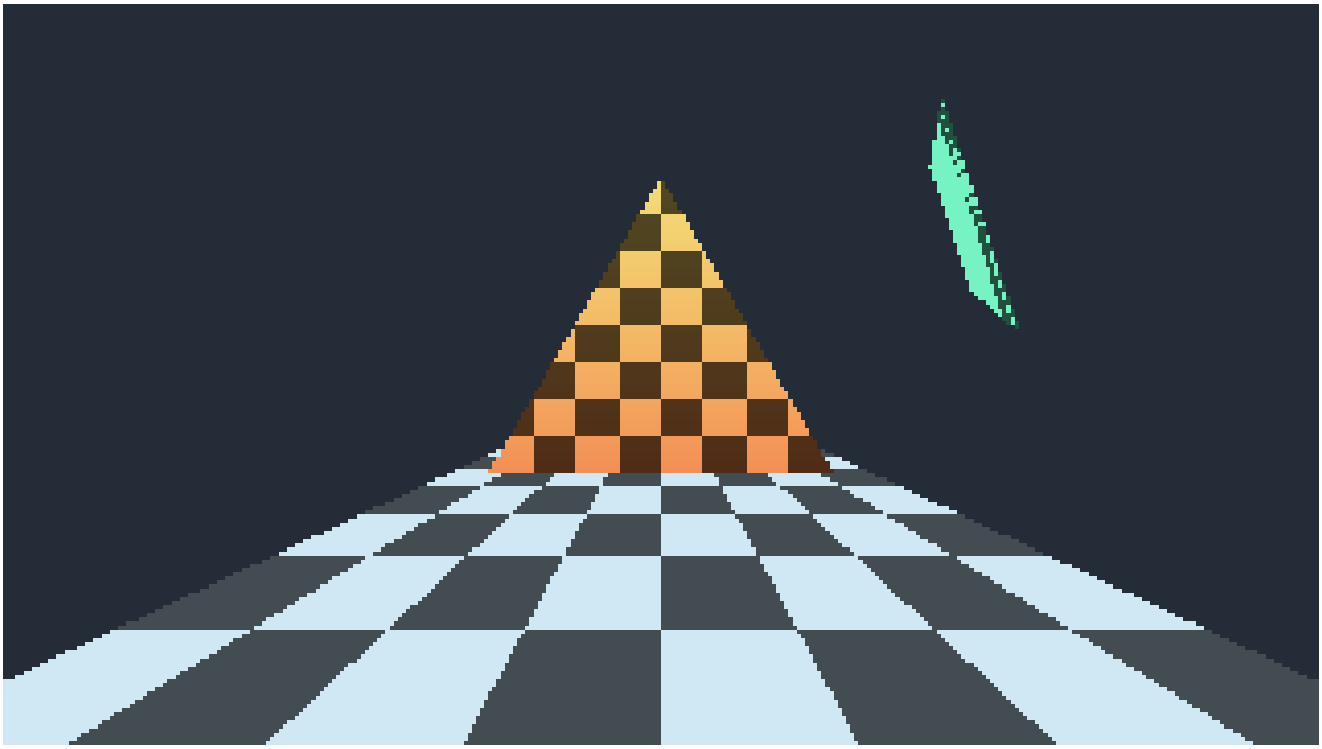
The supplied authoring environment was Linux x86-64 with Python3.13.5, GCC14.2.0, CMake and Node as recorded in validation/TESTED_ENVIRONMENT.json. The reference exercise suite passed123 unittest cases. The pipeline suite passed20 cases. C++ passed200 assertions in one CTest test executable, both in release and with address/undefined-behavior sanitizers. Twenty sample expressions were executed. The deliberately incomplete21.1 starter was confirmed to fail rather than silently importing its reference.

The cooker ran, emitted content-addressed files and the C++ renderer loaded the cooked mesh. The fixed sphere experiment compared1,000 rays against256 spheres with identical hit results to its brute-force

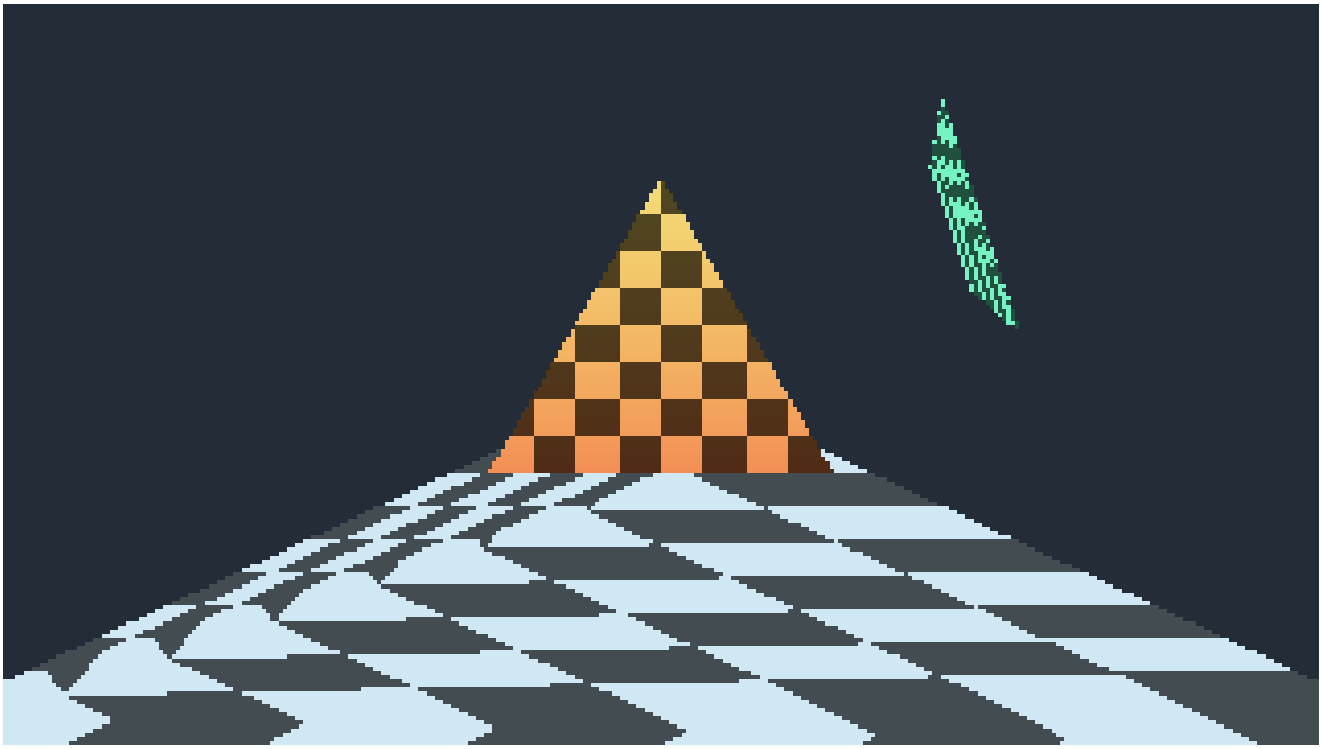
oracle. Included timings are local single-run CPU measurements only. JavaScript passed Node's syntax check; WGSL compilation and GPU execution were **not** performed. Native backend, device-loss, macOS/ Windows and independent reviewer validation are not claimed.

Use validation/ raw logs and evidence/ images/reports to inspect exactly what was tested. Re-run with validate_all.py on your own environment and keep fresh output separate. Do not replace source-review confidence with a fabricated hardware pass.

Numerical image evidence



CPU reference output: near-plane clipping, visibility and perspective-correct attributes. The checker is intentionally point sampled; this is not a completed PBR or antialiasing implementation.



Controlled fault: affine UV/color interpolation on the same scene and resolution. Compare the receding floor. Raw PPMs, encoded-domain differences and local timing scopes are in evidence/.

Sources and provenance

How this edition relates to the roadmap

Package 06 expands Volume 05 of the September 28, 2026 Computer Science and Computer Engineering for Games manifest. The source manifest and original curriculum were inspected directly. Module/lesson IDs, titles, learning outcomes, original lab assignments, original reading assignments and the first three lesson quiz questions are retained. The additional teaching, worked derivations, procedures, two application questions per lesson, module reviews and code in this bundle are original additions.

The canonical lesson questions were transcribed from the original curriculum PDF's Volume 05 section, not inferred from the shorter manifest. Original assessment examples are retained where a question refers to an example. Answer explanations are expanded. The complete original program and textbooks are not repackaged here.

Reading IDs R11 and R19-R27/R37 retain their original program meanings. S01-S16 are supplementary references for this volume only. Technical source links below were consulted or retrieved on September 28, 2026; S10 is specifically marked as a link-level check. API documentation can change. Record the version actually used in your own lab; this edition does not certify any unspecified browser, SDK, driver or engine release.

Canonical assigned references

R11 — C++ Core Guidelines editors: C++ Core Guidelines

Resource safety, interfaces, classes and concurrency; guidance rather than the normative language standard.

https://isocpp.github.io/CppCoreGuidelines/CppCoreGuidelines

R19 — Carnegie Mellon University: 15-462/662 Computer Graphics

Rendering, geometry and animation course structure and exercises.

https://15462.courses.cs.cmu.edu/fall2021/

R20 — Carnegie Mellon University: 15-466/666 Computer Game Programming

Runtime systems and game-production programming.

https://graphics.cs.cmu.edu/courses/15-466/

R21 — Khronos Group: Vulkan tutorial

Explicit graphics API study; feature-check the device before using the moving latest tutorial.

https://docs.vulkan.org/tutorial/latest/00_Introduction.html

R22 — SDL project: SDL3 documentation

Windowing, input, audio and platform services; do not mix SDL2 and SDL3 examples.

https://wiki.libsdl.org/SDL3/FrontPage

R23 — Matt Pharr, Wenzel Jakob, Greg Humphreys: Physically Based Rendering, fourth edition

Rendering mathematics and implementation; use chapter titles in lesson assignments.

https://www.pbr-book.org/4ed/contents

R24 — Microsoft: Direct3D 12 graphics

Windows graphics backend reference.

https://learn.microsoft.com/en-us/windows/win32/direct3d12/direct3d-12-graphics

R25 — Apple: Metal developer resources

Apple-platform graphics and compute reference.

https://developer.apple.com/metal/

R26 — Epic Games: Programming with C++ in Unreal Engine

Engine comparison after C++ foundations, not a beginner substitute.

https://dev.epicgames.com/documentation/en-us/unreal-engine/programming-with-cplusplus-in-unreal-engine

R27 — Glenn Fiedler: Fix Your Timestep!

Original practitioner explanation of fixed-step simulation; read critically and implement your own lab.

https://gafferongames.com/post/fix_your_timestep/

R37 — NVIDIA: CUDA Programming Guide

GPU execution and memory; CUDA is an optional vendor-specific lab route.

https://docs.nvidia.com/cuda/cuda-programming-guide/

Supplemental primary sources

S01 — W3C: WebGPU Shading Language (WGSL)

Host-shareable layout, alignment, storage/uniform contracts; evolving specification.

https://www.w3.org/TR/WGSL/

S02 — Netpbm: PPM format specification

Strict PPM and the explicitly documented sRGB variant used in this package.

https://netpbm.sourceforge.net/doc/ppm.html

S03 — Khronos Group: glTF 2.0 specification

Material/color semantics and tangent handedness; glTF importing is assigned, not implemented here.

https://registry.khronos.org/glTF/specs/2.0/glTF-2.0.html

S04 — Pharr, Jakob and Humphreys: PBRT 4: Bounding Volume Hierarchies

Spatial bounds, construction and traversal.

https://www.pbr-book.org/4ed/Primitives_and_Intersection_Acceleration/Bounding_Volume_Hierarchies

S05 — Khronos Group: Vulkan Tutorial: Frames in flight

Frame contexts; distinguish frame indexing from presentation-image indexing.

https://docs.vulkan.org/tutorial/latest/03_Drawing_a_triangle/03_Drawing/03_Frames_in_flight.html

S06 — GPU for the Web Community Group: WebGPU explainer

Conceptual API overview; an explainer, not a substitute for the normative specification.

https://gpuweb.github.io/gpuweb/explainer/

S07 — Khronos Group: Vulkan Guide: Synchronization examples

Dependencies, stages, accesses and barriers.

https://docs.vulkan.org/guide/latest/synchronization_examples.html

S08 — Khronos Group: Vulkan Guide: Swapchain semaphore reuse

Presentation waits may outlive the graphics submission fence.

https://docs.vulkan.org/guide/latest/swapchain_semaphore_reuse.html

S09 — Microsoft: Using resource barriers in Direct3D 12

Resource state and hazard management; do not transliterate Vulkan flags.

https://learn.microsoft.com/en-us/windows/win32/direct3d12/using-resource-barriers-to-synchronize-resource-states-in-direct3d-12

S10 — Apple: Synchronizing CPU and GPU work

Official Metal starting point. URL retrieved; dynamic page body was not fully inspectable during authoring.

https://developer.apple.com/documentation/metal/synchronizing-cpu-and-gpu-work

S11 — SDL: SDL_GetWindowSizeInPixels

Drawable pixel extent can differ from logical window size.

https://wiki.libsdl.org/SDL3/SDL_GetWindowSizeInPixels

S12 — Khronos Group: Vulkan Tutorial: Swap chain recreation

Out-of-date/zero-extent paths and safe fence-reset sequencing.

https://docs.vulkan.org/tutorial/latest/03_Drawing_a_triangle/04_Swap_chain_recreation.html

S13 — Pharr, Jakob and Humphreys: PBRT 4: Radiometry

Radiometric quantities and their domains.

https://www.pbr-book.org/4ed/Radiometry,_Spectra,_and_Color/Radiometry

S14 — Pharr, Jakob and Humphreys: PBRT 4: Roughness using microfacet theory

Microfacet distributions and model assumptions.

https://www.pbr-book.org/4ed/Reflection_Models/Roughness_Using_Microfacet_Theory

S15 — Pharr, Jakob and Humphreys: PBRT 4: Image texture

Texture reconstruction, footprints and filtering.

https://www.pbr-book.org/4ed/Textures_and_Materials/Image_Texture

S16 — GPU for the Web Community Group: WebGPU samples

Optional companion examples; no external sample source is bundled.

https://webgpu.github.io/webgpu-samples/

Scope of evidence

The diagrams in this edition are numerical reference-renderer outputs, not external photographs or a claim of hardware-rendered results. All included mesh, image and world fixtures are original procedural teaching data; assets/source/PROVENANCE.md documents them. Code is original and standard-library based except for the platform APIs explicitly used in the optional browser bridge. No textbook chapters, proprietary SDK binaries, third-party game assets or font files are redistributed.

The small examples illustrate contracts, not all details of their parent fields. PBRT supplies deeper transport/acceleration study; platform documentation supplies exact native API rules. The course's broader undergraduate-to-graduate pathway remains self-study, not accreditation, a degree, or a guarantee of employment.